

MSME EDI Exchange

Technical Architecture of the Infrastructure

A canonical-core, profile-projecting, triple-entry document exchange network for Indian micro, small and medium enterprises. Aligned to GST e-invoice, e-way bill, ONDC/Beckn, OCEN and UN/EDIFACT without being owned by any of them.

Document	Engineering specification — full infrastructure
Version	1.0
Date	24 August 2026
Runtime	TanStack Start v1 · React 19 · Vite 7 · Cloudflare Worker edge runtime
Data plane	Managed Postgres with row-level security
Cryptography	WebCrypto — SHA-256 content addressing, ECDSA P-256 detached signatures
Status of claims	Every statement marked Built reflects code in the repository today

Contents

1	Executive summary
2	Standards evaluation — why not IUDX wholesale
3	System architecture
4	Canonical data model
5	Identity, keys and signatures
6	Exchange protocol
7	Profile / adaptor layer
8	Triple-entry ledger
9	Database design
10	Server surface and module boundaries
11	Integration and agent surface
12	Security model
13	Testing and verification
14	Roadmap and deferred work

1. Executive summary

MSME EDI Exchange is a network on which two registered Indian businesses exchange structured trade documents — purchase orders, tax invoices, dispatch advices, goods receipt notes, payment advices and credit notes — as signed JSON, and simultaneously book the resulting accounting event once, together, instead of twice, separately.

Three decisions define the infrastructure:

- **One canonical document model is the source of truth.** GST e-invoice, e-way bill, ONDC/Beckn, OCEN and UN/EDIFACT are treated as *projections* produced on demand by profile codecs. No external payload is ever stored as the record of truth, so a schema revision on any single rail is a codec change, not a migration.
- **Peer-to-peer signed envelopes, not a data hub.** Each participant holds its own keypair; a document travels inside a detached-signature envelope that the receiver verifies against the registry before anything is parsed or written. Non-repudiation is a property of the message, not of the platform hosting it.
- **A triple-entry shared journal.** An accepted document emits a single hash-chained journal element that both counterparties sign. Reconciliation stops being fuzzy matching between two private ledgers and becomes signature verification over one shared record — which is also what makes a receivable financeable over OCEN without a trust intermediary.

The onboarding cost for an MSME is a GSTIN, a generated keypair and one callback endpoint. Everything heavier — registry lookup, signing, projection, ledger chaining — runs server-side in the edge runtime and never reaches the browser.

What exists today

Component	Status	Where
Canonical model, Zod schemas, GST totals	Built	src/lib/canonical/
Content hashing, canonical JSON, P-256 sign/verify	Built	src/lib/crypto.ts
GSTIN mod-36 validation and state codes	Built	src/lib/gstin.ts
Participant registry, key rotation with overlap	Built	src/lib/registry.server.ts
Signed envelopes, idempotent delivery, loopback transport	Built	src/lib/exchange.server.ts
Triple-entry ledger element, chaining, verification	Built	src/lib/ledger/element.ts
GST e-invoice profile, both directions	Built	src/lib/profiles/gst-einvoice.ts
Postgres schema, RLS, grants, append-only triggers	Built	supabase/migrations/
Server functions, dashboard and composer UI	Built	src/lib/edi.functions.ts, src/routes/
HTTP transport between deployed nodes	Partial	interface in place, loopback default
e-way bill, OCEN, ONDC, EDIFACT profiles	Planned	—
Zoho connector, Tally bridge, agent manifest	Planned	—

2. Standards evaluation — why not IUDX wholesale

IUDX (India Urban Data Exchange) was evaluated first, as an existing Indian public-good exchange stack. It is an excellent design for what it targets: discovery of and subscription to open, largely one-directional data resources published by public bodies. That is a different problem from a bilateral, legally consequential trade document.

2.1 Fit scoring

Requirement	IUDX hub	ONDC / Beckn spine	Plain REST hub
Two-party negotiated transaction	Absent — publish/subscribe only	Native — the protocol is a bilateral message exchange	Hand-rolled per integration
Non-repudiation of a document	Bearer token authorises access; it does not bind the payload	Detached signature over the body, registry-backed key lookup	None by default
Alignment with GST e-invoice / e-way bill	No mapping	GST and OCEN are already Beckn-adjacent in India's stack	Neutral
Counterparty and capability discovery	Strong — the catalogue is the best part of IUDX	Registry, thinner than the IUDX catalogue	None
MSME onboarding cost	Full stack: Java services, Kafka, Zookeeper, Postgres, Redis, Elasticsearch	One HTTPS callback endpoint and a keypair	Low but bespoke
Fit for an ERP integration	Poor — no document semantics	Good — the document is the unit of exchange	Poor — no shared semantics

2.2 What was taken from IUDX

- **Catalogue-style resource registry.** Every trading party and document type is a discoverable, versioned resource with a stable identifier — implemented as the participant registry plus the profile list each participant advertises.
- **Capability-scoped access rather than blanket API keys.** Access is granted per resource per consumer with a short life, expressed here as GSTIN-scoped row-level security plus per-call authorisation, and later as scoped agent tokens.
- **The adaptor pattern.** Ingestion and egress are pluggable codecs, never one monolithic parser — this became the profile layer in section 7.

2.3 What was rejected

- The deployment footprint. A five-service Java and Kafka topology is unshippable as MSME-facing SaaS and unjustifiable for document volumes measured in thousands per month.
- Bearer-token-only authenticity. A token proves who called the API; it does not prove who authored the invoice, which is the only thing that matters in a dispute.
- Time-series and streaming primitives, which have no counterpart in trade documents.

Conclusion. The architecture is an ONDC/Beckn-shaped signed peer-to-peer message spine carrying an internal canonical document model, with IUDX's registry and capability ideas reimplemented lightly, and a Pacio-style triple-entry ledger as the settlement truth layer.

3. System architecture

3.1 Layer view

Figure 1 — Layers. Each layer depends only on the layer beneath it.

6	APPLICATION & AGENT SURFACE
	composer . inbox/outbox . ledger board . profile export . manifest
5	TRIPLE-ENTRY LEDGER (Pacio-style shared journal)
	one signed element per event . hash chain . derived status
4	EXCHANGE LAYER
	signed envelope . submit -> ack -> on_response . idempotency . retry
3	PROFILE / ADAPTOR LAYER
	toProfile() / fromProfile() per standard, round-trip tested
2	CANONICAL DOCUMENT LAYER
	Zod-validated JSON . content-hashed . append-only revisions
1	IDENTITY & REGISTRY
	GSTIN anchor . P-256 keypair . published JWK . rotation overlap

3.2 Deployment view

Figure 2 — Deployment. Private keys never leave the server boundary.

BROWSER (React 19)	EDGE WORKER RUNTIME
routes/	*.functions.ts (thin RPC)
index auth app	
supabase client (RLS)	typed v
	*.server.ts (server-only)
	registry . exchange . queries
RLS-scoped reads	
v	admin signing keys
POSTGRES	participants . participant_private_keys
	canonical_documents . document_revisions . envelopes
	ledger_elements . ledger_signatures . ledger_disputes
	profile_submissions
	^
	signature verified BEFORE parse
routes/api/public/edi/*	inbound peer traffic
(webhooks, counterparty delivery, agents)	

The browser holds no signing material. Every operation that touches a private key, the registry's privileged view, or the ledger chain head runs inside a server function; the client receives only read models shaped for display.

3.3 Trust boundaries

Boundary	Crossed by	Control
Browser → server function	Authenticated user actions	Session bearer token attached by client middleware; the caller's identity is resolved server-side and never trusted from the request body

Boundary	Crossed by	Control
Server function → database	All reads and writes	Row-level security scoped to the GSTINs the caller owns; the privileged client is used only after ownership has been verified
Peer node → public API route	Inbound documents and webhooks	Detached signature verified against the registry before any parse or write
Server → external rail (IRP, ERP)	Profile submissions	Adapter interface with a mock default; credentials held as server secrets

4. Canonical data model

The canonical document is a plain JSON object, validated by Zod, content-addressed by SHA-256 over a deterministic serialisation, and versioned by an append-only revision chain. TypeScript types and runtime validation share one definition; there is no second hand-written interface to drift.

4.1 CanonicalDocument

Field	Type	Notes
id	string (uuid)	Stable internal identifier, the suffix of the document's IRI
kind	ORDER INVOICE DISPATCH GRN PAYMENT_ADVICE CREDIT_NOTE	Drives which party is the issuer and which ledger legs are produced
documentNumber	string	Issuer's own numbering, carried through to every profile
issueDate	string (ISO date)	Reformatted per profile, e.g. DD/MM/YYYY for the IRP
currency	string	INR by default
seller / buyer	Party	Full snapshot at issue time, not a registry reference — the document must stay self-describing years later
lines	LineItem[]	At least one line; totals are always derived, never stored as input
transport	Transport?	Mode, vehicle, transporter GSTIN, distance — the e-way bill inputs
references	{ orderRef?, invoiceRef?, reasonCode? }	Links a document to its antecedent; reasonCode carries credit-note grounds
paymentTermsDays	number	Feeds receivable ageing and any financing projection
notes	string?	Free text, never machine-interpreted
revision	number	Starts at 0; each accepted amendment appends
contentHash	string (hex)	SHA-256 over the canonicalised body, set on persist
createdAt	string (ISO datetime)	

4.2 Party, LineItem, Transport

Party	gstIn, legalName, tradeName?, stateCode, address, city, pincode, email?, phone?
LineItem	id, description, hsn, quantity, uom, unitPrice, discount, taxRate, cessRate?
Transport	mode (ROAD RAIL AIR SHIP), vehicleNumber?, transporterGstin?, distanceKm?, dispatchFrom?, shipTo?

4.3 Derived totals

Totals are computed, never accepted from the caller, so a projection can never disagree with the document it came from. Intra-state supply (matching state codes) splits into CGST and SGST at half the rate each; inter-state produces IGST at the full rate. Cess applies on the same taxable value. Every monetary result is rounded to two decimals at the point of computation, not at display.

```
taxable(line) = round2(quantity * unitPrice - discount)
intraState    = seller.stateCode === buyer.stateCode

intra:  cgst += taxable * rate/200      inter:  igst += taxable * rate/100
        sgst += taxable * rate/200
cess    += taxable * cessRate/100

grandTotal = taxableValue + cgst + sgst + igst + cess
```

4.4 Validation contract

A single Zod schema is the validation authority for composer input, server-function input and inbound peer payloads alike. GSTIN is checked twice: structurally by regular expression, then arithmetically by the mod-36 check character (section 5.1). Line arrays must be non-empty, rates non-negative, pincodes six digits, and a party's state code must equal the first two characters of its GSTIN.

4.5 Revisions and content addressing

Canonicalisation sorts object keys recursively and drops undefined values, so two structurally identical documents serialise to identical bytes and therefore to identical hashes on any machine. A revision records the previous content hash alongside the new one, giving each document its own miniature chain; an amendment can be replayed and verified without trusting the row it is stored in.

revision 0	prevHash = null	contentHash = H(body0)
revision 1	prevHash = H(body0)	contentHash = H(body1)
revision 2	prevHash = H(body1)	contentHash = H(body2)

5. Identity, keys and signatures

5.1 GSTIN as the identity anchor

A participant is identified by its GSTIN, which already carries a state code, a PAN and a checksum — so identity is partially verifiable offline before any network call. Validation runs the mod-36 algorithm over the first fourteen characters using the digit-then-uppercase alphabet and compares the computed character with the fifteenth.

```
value(c) = 0-9 for '0'-'9', 10-35 for 'A'-'Z'
w        = value(c) * ( 2 if position is even else 1)      -- 1-indexed
digit     = sum over positions of ( floor(w/36) + w mod 36 )
check     = alphabet[ (36 - digit mod 36) mod 36 ]
```

The database enforces the structural pattern as a column constraint; the application enforces the checksum. Both must pass before a participant row exists.

5.2 Key material

Property	Value
Algorithm	ECDSA over P-256 with SHA-256 (WebCrypto — available in browsers and in the Worker runtime)
Public key format	JWK, published in the registry — the shape ONDC uses for registry-backed lookup
Key identifier	First 16 hex characters of SHA-256 over the canonicalised public JWK — deterministic, no counter to coordinate
Private key storage	participant_private_keys, a table with no policies for authenticated or anonymous roles; reachable only by the privileged server client
Rotation	A new keypair becomes active; the previous key is appended to rotated_keys with its retirement timestamp, never deleted

5.3 Signing and verification

Signatures are detached and cover the content hash rather than the raw body, so the same signature verifies whichever transport or encoding carried the document. Verification tries the participant's active public key first, then each historical key inside its rotation overlap window, so a document signed moments before a rotation still verifies afterwards.

```
sign:    digest    = SHA-256( canonicalize(body) )
         signature = base64( ECDSA-P256-SHA256(privateKey, digest) )

verify:  candidates = [ active key ] + [ rotated keys within the overlap window ]
         accept on the first candidate for which verify(key, digest, signature) holds
```

5.4 Algorithm-agnostic seam

Three rails demand three signature families: internal exchange uses P-256, ONDC/Beckn uses Ed25519, and the GST IRP uses RSA with X.509 certificates. Rather than letting two signing paths coexist undeclared, signing sits behind a narrow interface with P-256 as the internal default; a profile needing another family supplies its own implementation of the same interface.

```
interface Signer {  
  keyId: string  
  alg: 'ES256' | 'EdDSA' | 'RS256'  
  sign: (digestHex: string) => Promise<string>  
  verify: (digestHex: string, signature: string) => Promise<boolean>  
}
```

6. Exchange protocol

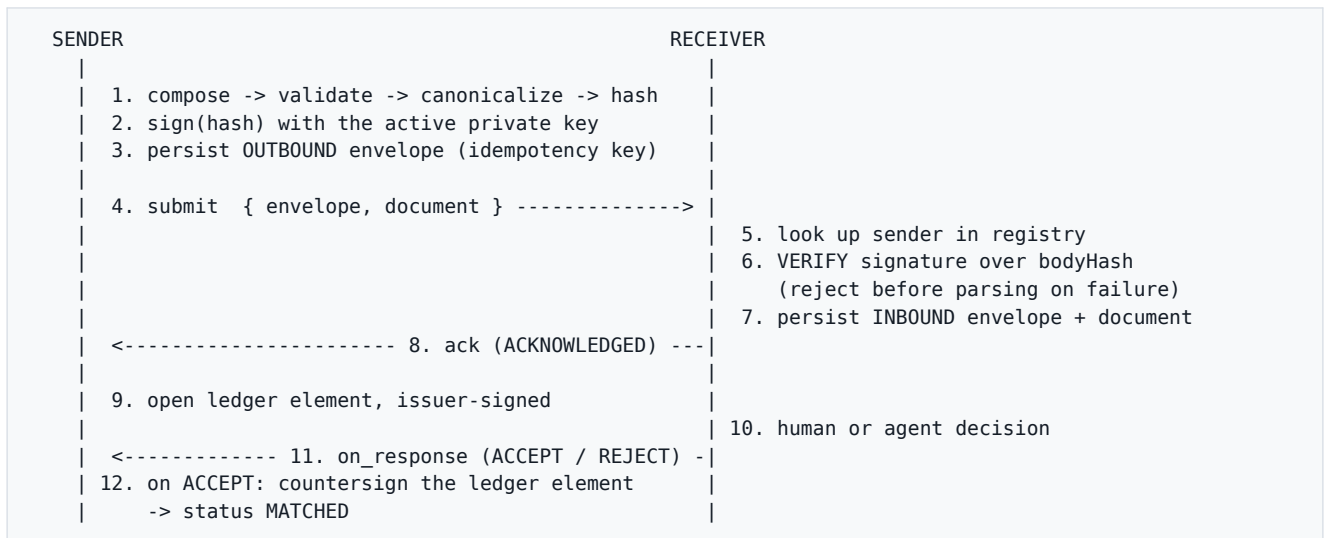
A document is never delivered bare. It travels inside an envelope naming the sender, the receiver, the action, the body hash, the signature and the key that produced it. The envelope is the unit of retry, idempotency and audit; the document is the unit of meaning.

6.1 Envelope

Field	Purpose
id	Envelope identity, distinct from the document's
direction	OUTBOUND on the sender's node, INBOUND on the receiver's
documentId, action	What is being conveyed — issue_invoice, accept, reject
fromGstin, toGstin	Registry keys for signature verification and routing
bodyHash	SHA-256 of the canonicalised document body; the signed value
signature, signerKeyId	Detached signature and the key that produced it
idempotencyKey	Unique per (document, revision, action) — database-enforced
status, attempts, transitions	Current state, delivery attempts and the full timestamped state history
deadLettered, lastError	Terminal failure marker and the last transport error, kept for inspection

6.2 Sequence

Figure 3 — submit → ack → on_response. The ledger opens on send and closes on accept.



6.3 Idempotency and retries

The idempotency key is derived, not random: documentId : revision : action. A duplicate send therefore collides on a unique index and returns the existing envelope instead of producing a second delivery or a second ledger element. Retries increment the attempt count and append a transition entry; a delivery that exhausts its attempts is dead-lettered rather than silently dropped, and stays inspectable with its last transport error attached.

6.4 State machine

```
DRAFT --send--> SENT --peer receipt--> ACKNOWLEDGED ---accept--> ACCEPTED
      |                                     |
      |                                     +--reject--> REJECTED
      +--transport exhausted--> FAILED (dead-lettered)
```

Only ACCEPTED closes a ledger element into MATCHED. REJECTED leaves the element open and pending, deliberately: a rejection is itself a fact both parties should be able to see and prove.

6.5 Transport

Delivery is pluggable behind one interface. The default is an in-process **loopback** transport that mirrors the envelope into the counterparty's inbox on the same node — after running the identical registry lookup and signature verification a remote peer would run. The whole protocol is therefore demonstrable and testable in a single environment with no check weakened. HTTP delivery to a peer's published endpoint implements the same interface, so going from one node to two is a configuration change rather than a protocol change.

7. Profile / adaptor layer

A profile is a bidirectional codec between the canonical document and one external standard. Every profile exposes the same functions, and every profile is round-trip tested: project a canonical document out, read it back in, and assert the recoverable fields survive.

```
toProfile(canonical)    -> external payload           // export / submission
fromProfile(payload)    -> Partial<CanonicalDocument> // ingestion
validateFor(canonical) -> string[]                  // rail-specific pre-flight errors
droppedOnExport         : string[]                  // fields the rail cannot carry
```

7.1 GST e-invoice (NIC schema 1.1) — built

Canonical	IRP payload	Transformation
kind	DocDtls.Type	INVOICE → INV, CREDIT_NOTE → CRN
documentNumber	DocDtls.No	verbatim
issueDate	DocDtls.Dt	ISO date → DD/MM/YYYY
seller	SellerDtls	Gstin, LglNm, TrdNm, Addr1, Loc, Pin (numeric), Stcd, Em, Ph
buyer	BuyerDtls	as seller, plus Pos (place of supply) from the buyer's state code
lines[]	ItemList[]	SINo, PrdDesc, HsnCd, Qty, Unit, UnitPrice, TotAmt, Discount, AssAmt, rate and tax amounts, cess
derived totals	ValDtls	AssVal, CgstVal, SgstVal, IgstVal, CesVal, TotInvVal
transport	EwbDtls	vehicle, transporter GSTIN, distance — populated only when transport is present

Pre-flight validation reports rail-specific problems the canonical schema tolerates — missing HSN, absent pincode, a document kind the IRP has no type code for — before any submission is attempted.

Dropped on export is published, not hidden: fields the IRP schema cannot carry (internal identifiers, revision and content hash, payment terms, free-text notes, non-invoice document kinds) are listed explicitly — which is precisely why the canonical document, not the IRP payload, remains the record of truth.

Inverse projection reads an IRP payload back into a partial canonical document, so an invoice that entered the world through an accounting package's IRP integration can still be adopted onto the network.

7.2 Submission adapters

Submitting to a live IRP requires GSP credentials. Submission is therefore an adapter with a **mock** implementation as the default: it validates, projects, records the attempt in `profile_submissions` with a synthetic reference number, and returns the shape a live adapter returns. Enabling the sandbox or production IRP is a configuration swap rather than a code-path change, and no demo blocks on credentials.

7.3 Planned profiles

Profile	Shape	Status	Note
e-way bill	Codec over transport plus invoice value	Planned	Canonical Transport already carries every required input
OCEN	Loan application projection over receivables	Planned	Consumes MATCHED ledger elements as collateral evidence

Profile	Shape	Status	Note
ONDC / Beckn	State machine over a transaction id, not a codec	Planned	Deliberately not shaped like a profile — needs its own design pass
UN/EDIFACT D96A INVOIC / ORDERS	Segment-oriented flat file codec	Planned	Highest fixture burden, lowest near-term Indian MSME demand

8. Triple-entry ledger

Conventional double entry produces two private books that must later be matched, which is where most MSME reconciliation cost and most invoice disputes live. The Pacio-style model produces **one** journal element per trade event, signed by both counterparties and chained to its predecessor. Both parties derive their books from the same element, so reconciliation becomes verification rather than matching.

8.1 Element construction

```
event = { documentId, documentKind, documentHash,
          sellerGstin, buyerGstin, debit, credit,
          tax: { cgst, sgst, igst, cess, total } }

eventHash = SHA-256( canonicalize(event) )
prevHash = chainHash of this participant's most recent element ( '' for the first )
chainHash = SHA-256( prevHash + ':' + eventHash )
```

Because the event hash covers the document's own content hash, an element is bound to an exact document revision. Altering the document afterwards breaks the binding; altering the element breaks the chain.

8.2 Account legs by document kind

Document kind	Debit	Credit
INVOICE	Buyer — Purchases	Seller — Sales Revenue
CREDIT_NOTE	Seller — Sales Returns	Buyer — Purchase Returns
PAYMENT_ADVICE	Seller — Bank	Buyer — Bank
ORDER, DISPATCH, GRN	Buyer — Commitments	Seller — Commitments

Non-financial documents still produce an element, booked against a commitments account. They carry no profit-and-loss effect but they do carry evidence — a signed, timestamped record that an order was placed or goods were received, which is what a lender or an arbitrator actually needs.

8.3 Derived status

Status is never stored and edited; it is derived from the signatures and disputes on record, which is what makes an append-only table sufficient.

Condition	Status
A dispute row exists for the element	DISPUTED
Two or more distinct GSTINs have signed	MATCHED
Otherwise	PENDING_COUNTERSIGN

8.4 Append-only enforcement

Immutability is enforced at the database in two independent layers, because a grant alone is not enough when privileged server code also connects:

- No UPDATE or DELETE privilege on ledger_elements, ledger_signatures or ledger_disputes is granted to any role — including the service role.
- A BEFORE UPDATE OR DELETE trigger on each of those tables raises an exception naming the rejected operation, so mutation fails even for a caller that somehow held the privilege.

- Corrections are made by appending a reversing element, never by editing history.

8.5 Chain verification

The ledger view replays every element in sequence, recomputing each chain hash from the recorded previous hash and event hash. It reports either a clean chain or the index of the first element whose link does not hold — a tamper indicator surfaced in the interface rather than buried in a log.

```
expectedPrev = ''
for each element in sequence order:
    if element.prevHash != expectedPrev:                -> broken here
    if SHA-256(prevHash + ':' + eventHash) != chainHash: -> broken here
    expectedPrev = element.chainHash
```

8.6 Disputes

A counterparty that disagrees appends a dispute row carrying the raiser's GSTIN, a reason and a timestamp. The element itself is untouched; its derived status becomes DISPUTED. The dispute is as permanent and as provable as the claim it contests.

8.7 Why this is the financing primitive

A MATCHED element is a receivable the debtor has cryptographically acknowledged — materially stronger collateral than an invoice copy, and the artefact presented over OCEN in the financing flow. Per-element proofs will eventually let a borrower prove a single receivable to a lender without exposing the counterparty's wider ledger.

9. Database design

Postgres with row-level security on every table in the public schema, explicit grants per role, and security-definer helper functions so document policies do not recurse through the participants table's own policies.

9.1 Tables

Table	Purpose	Key columns	Write mode
participants	Registry of trading parties	gstin (PK), user_id, legal_name, state_code, endpoint, key_id, public_key_jwk, rotated_keys, profiles, status	Owner CRUD
participant_private_keys	Server-only signing material	(gstin, key_id) PK, private_key_jwk, active	Privileged only
canonical_documents	Current state of each document	id, kind, document_number, seller_gstin, buyer_gstin, seller, buyer, lines, transport, revision, content_hash, status, direction, totals	Party CRUD; delete drafts only
document_revisions	Append-only revision chain	document_id, revision, prev_hash, content_hash, body, author_gstin	Insert only
envelopes	Signed message envelopes	document_id, direction, from_gstin, to_gstin, action, body_hash, signature, signer_key_id, idempotency_key (unique), status, attempts, transitions	Insert and status update
ledger_elements	Shared journal elements	sequence, document_id, event_hash (unique), prev_hash, chain_hash, debit, credit, tax, narrative	Insert only, trigger-enforced
ledger_signatures	Per-party countersignatures	element_id, gstin, signature, key_id	Insert only, trigger-enforced
ledger_disputes	Raised disputes	element_id, raised_by, reason	Insert only, trigger-enforced
profile_submissions	External rail submissions	document_id, profile, status, reference_number, payload, response, adapter	Party insert and update

9.2 Access helpers

Ownership is answered by a security-definer function, so a policy on any table can ask “does the caller control this GSTIN?” without triggering the participants table's own row-level security and recursing.

```
is_my_gstin(_gstin text) returns boolean
language sql stable security definer set search_path = public
select exists ( select 1 from public.participants
               where user_id = auth.uid() and gstin = _gstin )

ledger_chain_head(_gstin text) returns text
-- most recent chain_hash on either side of this participant's ledger, '' when none
```

9.3 Policy pattern

Table	SELECT	INSERT	UPDATE / DELETE
participants	Any signed-in user — the registry is a directory	user_id = auth.uid()	Owner only

Table	SELECT	INSERT	UPDATE / DELETE
canonical_documents	Either party's GSTIN is mine	Either party is mine	Update: either party. Delete: drafts only
document_revisions	Through the parent document	author_gstin is mine	Not granted
envelopes	Sender or receiver is mine	Sender is mine	Update: either party. Delete not granted
ledger_* tables	Either party is mine	Either party is mine	Not granted; the trigger raises
participant_private_keys	No policy — unreachable from the browser by design	—	—

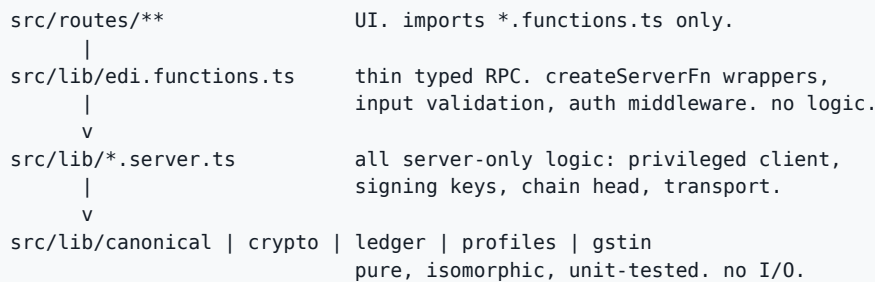
9.4 Grants

Grants are written in the same migration as the table, immediately after creation and before row-level security is enabled, because policies alone confer no privilege. No table grants anything to the anonymous role: the entire surface is authenticated. The ledger tables grant only SELECT and INSERT — to the authenticated role and to the service role alike.

10. Server surface and module boundaries

10.1 Module layering

Figure 4 — One direction of dependency. Pure modules know nothing about the database.



Files named `*.server.ts` are excluded from client bundles by build-time import protection, so a private-key path cannot reach the browser through a shared helper. Server-function modules stay thin wrappers by rule: module scope holds only imports, erased types and exported declarations, because code splitting removes runtime siblings from those files.

10.2 Server functions

Function	Method	Does
registerParticipant	POST	Validates the GSTIN, generates a keypair, publishes the public JWK, stores the private key server-side
rotateKey	POST	Issues a new keypair and retires the old one into the overlap window
myParticipants / directory	GET	The caller's own GSTINs; the network directory for choosing a counterparty
createDraft	POST	Validates, computes totals and the content hash, writes revision 0
listMyDocuments / getDocumentDetail	GET	Read models: inbox and outbox summaries, full detail with envelopes and revisions
sendDocumentFn	POST	Signs, persists the envelope idempotently, delivers, opens the ledger element
respondFn	POST	Accepts or rejects an inbound document; countersigns the element on accept
ledger	GET	Chained elements with derived status and a chain verification result
disputeElement	POST	Appends a dispute against an element
gstProjection	GET	Projects a document to the IRP payload for preview or download

10.3 Public HTTP routes

Only traffic originating outside the app uses raw HTTP routes, placed under `src/routes/api/public/edi/*`. That prefix bypasses site authentication, so each handler authenticates its own caller. The ordering inside every handler is fixed:

1. read the raw body as text (never parse first)
2. resolve the claimed sender in the registry
3. verify the detached signature over the body hash
4. reject with 401 on any failure
5. only now parse, validate with Zod, and write

11. Integration and agent surface

The network is only useful to an MSME if it meets the books where they already are — which in India means Tally on a desktop and Zoho in a browser — and only useful to a fintech if a machine can transact on it. Both are the same problem: a codec plus a scoped credential.

11.1 Unified connector interface

```
interface Connector {
  id:      'erp-zoho' | 'erp-tally' | string
  pull:    (since: string) => Promise<Partial<CanonicalDocument>[]>
  push:    (doc: CanonicalDocument) => Promise<{ externalId: string }>
  reconcile: (elements: LedgerElement[]) => Promise<void>
}
```

Every connector maps to and from the canonical document, never to another connector. Adding the n-th accounting package is one codec, not n integrations.

11.2 Zoho — planned

OAuth is not hand-rolled. The platform's per-app-user connector owns the consent flow and token storage, so the connector module contains only the codec and the sync logic and never holds a refresh token. Each app user connects their own Zoho organisation; the network never sees a shared credential.

11.3 Tally — cloud half in scope

Tally speaks XML over a local HTTP port on a desktop, so the agent that talks to it is a separate binary in a separate repository and cannot be built here. What this codebase owns is the cloud half of the bridge, which is the part that has to be right:

- A pairing endpoint binding a desktop installation to a participant GSTIN.
- A scoped agent token, capability-limited to that GSTIN's documents — in the IUDX spirit.
- The erp-tally XML codec, running server-side and unit-testable without a desktop.
- A push/pull queue the agent drains, so an offline desktop delays sync rather than losing it.

11.4 Fintech AI agents

A payments or lending platform integrates by reading a machine-readable capability manifest and then calling the same authenticated surface a human uses. Two properties make this safe: authorisation is per call and resolved from the caller's verified token, and every agent action produces the same signed envelope and ledger element a human action produces — so agent activity is as auditable and as non-repudiable as anything else on the network.

Surface	For	Status
/capabilities manifest	Discovery of document kinds, profiles and endpoints by an agent	Planned
MCP server	One server whose tools resolve the caller from a verified OAuth token; scoping is per call, not per instance	Planned
Scoped agent tokens	Capability-limited, GSTIN-scoped, short-lived	Planned

12. Security model

Threat	Control
Replay of a captured envelope	Idempotency key derived from document, revision and action and enforced by a unique index — a replayed envelope resolves to the existing record and produces no second delivery and no second ledger element
Forged sender or spoofed document	Detached signature over the content hash, verified against the registry's published JWK before any parse; a body failing verification is never written
Key compromise	Rotation issues a new keypair and retires the old one into a bounded overlap window; historical keys still verify past signatures but cannot sign new ones. Private keys live in a table with no policies for authenticated roles
Cross-tenant data leakage	Row-level security on every table, scoped through a security-definer ownership function; the privileged client is used only inside server modules and only after ownership has been checked against the caller's session
Ledger tampering	No UPDATE or DELETE privilege on any ledger table for any role, plus BEFORE UPDATE OR DELETE triggers that raise; hash chaining makes any out-of-band mutation detectable by replay
Document altered after acceptance	The ledger event hash binds the document's content hash, so an amended document no longer matches the element both parties signed
Credential exposure in the browser	Signing keys, privileged database credentials and rail credentials are read only inside server-function handlers; *.server.ts modules are excluded from client bundles
Malicious inbound payload	Signature verified first, then Zod validation before any persistence; totals are always recomputed rather than trusted from the payload
Unauthorised agent action	Agent tokens are capability-scoped to a GSTIN and resolved per call; every agent action leaves the same signed, chained evidence as a human action

Residual gaps, stated rather than hidden. There is no rate-limiting primitive on this runtime, so abuse throttling on public endpoints is custom work that has been deferred. Selective disclosure over ledger proofs is not built, so a lender today sees whole elements rather than minimal proofs. Live IRP submission runs against a mock adapter until GSP credentials exist.

13. Testing and verification

Thirty-two unit tests across four suites guard the parts where a silent error would be expensive: determinism, cryptography, tax arithmetic and chain integrity. Every test runs without a database or a network.

Suite	Tests	Guards
canonical.test.ts	11	Canonicalisation determinism and key-order independence, content hashing, revision chaining, signature round-trip, tamper detection, wrong-key rejection
ledger/element.test.ts	9	Event hash construction, account legs per document kind, chain linking, tamper detection by replay, derived status transitions
profiles/gst-einvoice.test.ts	6	Projection of single-line, inter-state and discount-plus-cess documents; round trip through the inverse projection; pre-flight validation
gstin.test.ts	6	Mod-36 check character over known-good and corrupted GSTINs, structural pattern, state code extraction

A note on signature assertions. ECDSA P-256 is non-deterministic — signing the same digest twice yields different bytes. Tests therefore assert that verification succeeds and that tampering makes it fail, never that a signature equals a fixed byte string, and use a committed fixed test keypair for reproducibility.

Beyond unit tests, the loopback transport makes the entire protocol exercisable end to end in a single environment: two test participants, a real signature, a real registry lookup, a real verification and a real chained ledger — with no check weakened for the sake of the test.

14. Roadmap and deferred work

14.1 Execution order

#	Stage	Status
1	Canonical model, crypto, GSTIN, test suite	Built
2	Participant registry, keypairs, rotation	Built
3	Database schema, RLS, grants, append-only triggers	Built
4	Exchange with loopback transport, composer and dashboard	Built
5	Triple-entry ledger, reconciliation board, disputes	Built
6	GST e-invoice profile with mock IRP submission	Partial
7	HTTP transport between deployed nodes	Planned
8	Zoho connector through per-app-user OAuth	Planned
9	Cloud-side Tally bridge; e-way bill profile	Planned
10	OCEN financing surface over MATCHED elements	Planned
11	Capability manifest and MCP server for agents	Planned

The ledger was deliberately built before the connectors. It is what makes this a settlement network rather than a document relay, and every connector written before ledger semantics landed would have been a codec facing revision.

14.2 Deliberately deferred

Item	Why it waits
Merkle selective-disclosure proofs	Real value, but it only matters once a counterparty wants to verify a receivable without trust. Ship the signed chained ledger first; add the proof endpoint when the first lender asks
UN/EDIFACT D96A	Highest fixture burden of any profile and the lowest near-term demand among Indian MSMEs. The profile interface stays ready; the codec is built on demand
Agent rate limiting	No rate-limiting primitive exists on this runtime, so it is bespoke work. The gap is recorded rather than papered over
ONDC / Beckn transaction flow	It is a state machine over a transaction id, not a codec, and forcing it into the profile folder's shape would be the wrong abstraction. It gets its own design pass

End of document.